

Explainability, Control and Trust in AI

Bridging Logic-Based Rules and Post-Hoc XAI (LIME & SHAP)

Rule-Based Logic

LIME

SHAP

The "Black Box" Problem in AI



The Trade-off

Higher model complexity (Deep Learning, Ensembles) yields accuracy but sacrifices interpretability.



The Stakes

In high-stakes domains, blind predictions are severe liabilities — medical, financial, legal.

High-Stakes Domains



Healthcare

Why diagnose a tumor?

Risk: Medical malpractice



Finance

Why reject a loan?

Risk: Regulatory/discrimination



Legal

Why flag this case?

What is XAI?

Explainable AI (XAI) — architectural patterns and tools used to make AI decisions human-understandable. Essential for debugging pipelines, compliance audits, and establishing user trust.

Two Paths to Explainability

PATH A — Intrinsic (By Design)

Models transparent by nature

Decision Trees

Linear Regression

Rule-Based Systems

✓ 100% transparent

✓ Fast & computationally cheap

✗ May miss complex non-linear patterns



Glass Box — Human readable by design

OR

PATH B — Post-Hoc (After the Fact)

External tools interpreting trained models

LIME

SHAP

Grad-CAM (for images)

✓ State-of-the-art model accuracy

✓ Works on any black-box model

✗ Computationally expensive approximations



Black Box + Magnifying Glass

Logic Programming & Symbolic Rules — Guardrails

Symbolic AI

Hard-coded knowledge, deterministic rules, always predictable.

Neuro-Symbolic

Combines pattern recognition (ML) with reasoning (rules).

Pipeline Guardrails

Rules act as safety filters before and after ML predictions.

```
def safe_loan_prediction(user_data, ml_model):  
    # 1. Symbolic Guardrail – Hard Constraint  
    if user_data['credit_score'] < 500:  
        return {"status": "Rejected",  
                "reason": "Policy: Min credit score"}  
  
    # 2. ML Model Prediction  
    prediction = ml_model.predict(user_data)  
  
    # 3. Post-Prediction Sanity Check  
    if prediction == 'Approved' \  
        and user_data['amount'] > 1_000_000:  
        return {"status": "Manual Review"}  
  
    return {"status": prediction}
```

Raw Data



Rule Engine



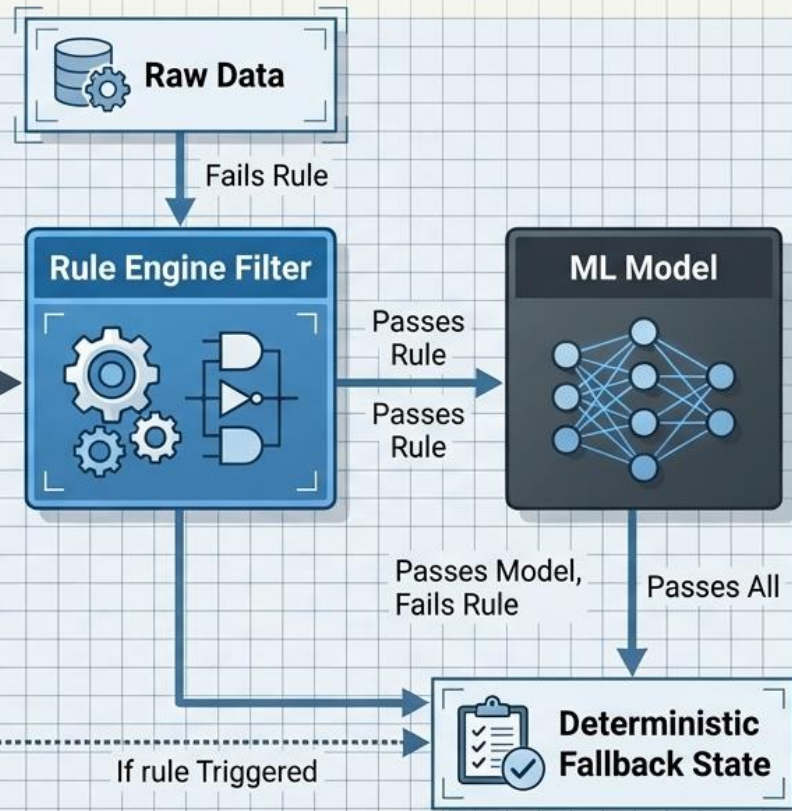
ML Model



Output

Symbolic Logic & Rule-Based Guardrails

```
1 def safe_loan_prediction(user_data, ml_model):
2
3     if user_data['credit_score'] < 500:
4         return 'Rejected: Policy Minimum'
5
6     prediction = ml_model.predict(user_data)
7
8     if prediction == 'Approved' and
9        user_data['requested_amount'] > 1000000:
10
11         return 'Manual Review Required'
12
13     return prediction
```



Transitioning to Post-Hoc Explainability

When system constraints require complex models, post-hoc XAI frameworks help explain them.



GLOBAL Explainability

"The Forest"

System-wide model behavior

"Which features generally drive this model?"

Used by:

- Data Engineers
- Architects
- Auditors / Compliance



LOCAL Explainability

"The Tree"

Single-prediction behavior

"Why was this specific user declined?"

Used by:

- Customer Support
- End-Users
- Legal Teams

Introduction to LIME

LIME

Local Interpretable Model-agnostic Explanations

Focus

Strictly local explainability

Approach

Model-agnostic — works with any `.predict()` method

Mechanism

Perturbs input, observes prediction changes, fits a local linear model

```
import lime.lime_tabular

# 1. Initialize the Explainer
explainer = lime.lime_tabular.LimeTabularExplainer(
    training_data = X_train.values,
    feature_names = X_train.columns,
    mode = 'classification'
)

# 2. Explain a single prediction
exp = explainer.explain_instance(
    data_row = X_test.iloc[0].values,
    predict_fn = rf_model.predict_proba
)

# 3. Show explanation
exp.show_in_notebook()
```

Introduction to SHAP

SHAP

SHapley Additive exPlanations

Focus

Both local AND global explainability

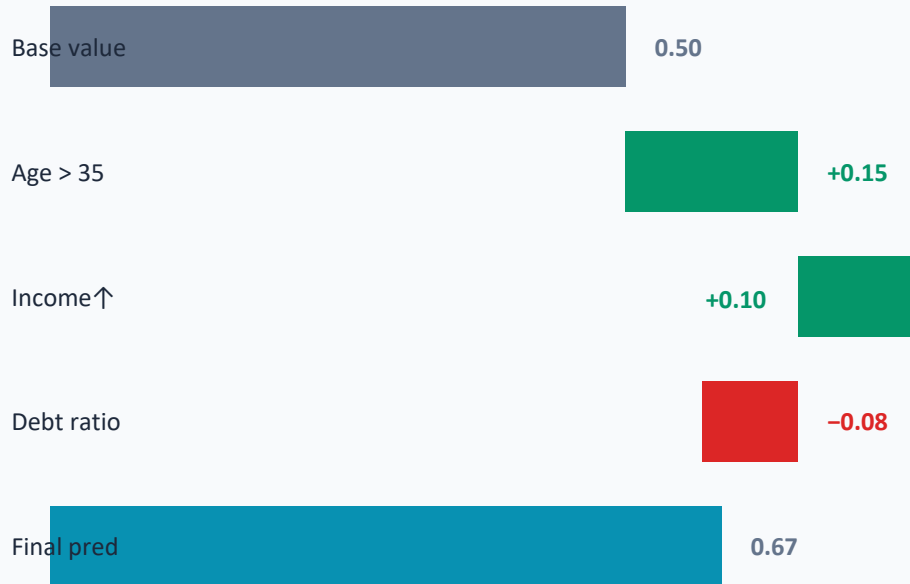
Foundation

Cooperative game theory (Shapley values)

Goal

Mathematically fair contribution of each feature vs. baseline average

SHAP Waterfall Plot (Conceptual)



The Game Theory Behind SHAP



The Game

The model's prediction



The Players

Features: Age, Income, Credit Score...



The Payout

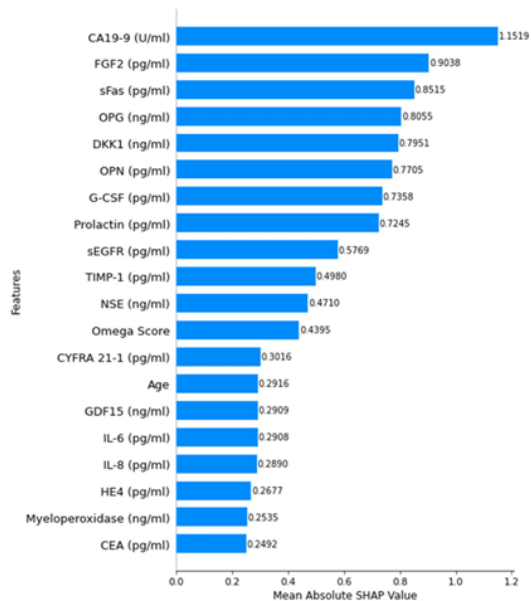
Prediction score difference from baseline

Key Math:

SHAP calculates the marginal contribution of each feature across ALL possible feature combinations — then fairly distributes the total payout. Values are additive: they sum exactly to the difference between actual and baseline predictions.

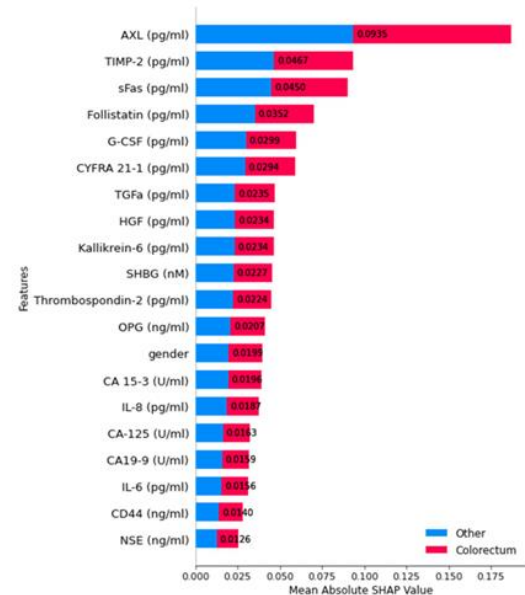
Additive Property:





SHAP summary plot for feature importance at classification level 1

```
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
```

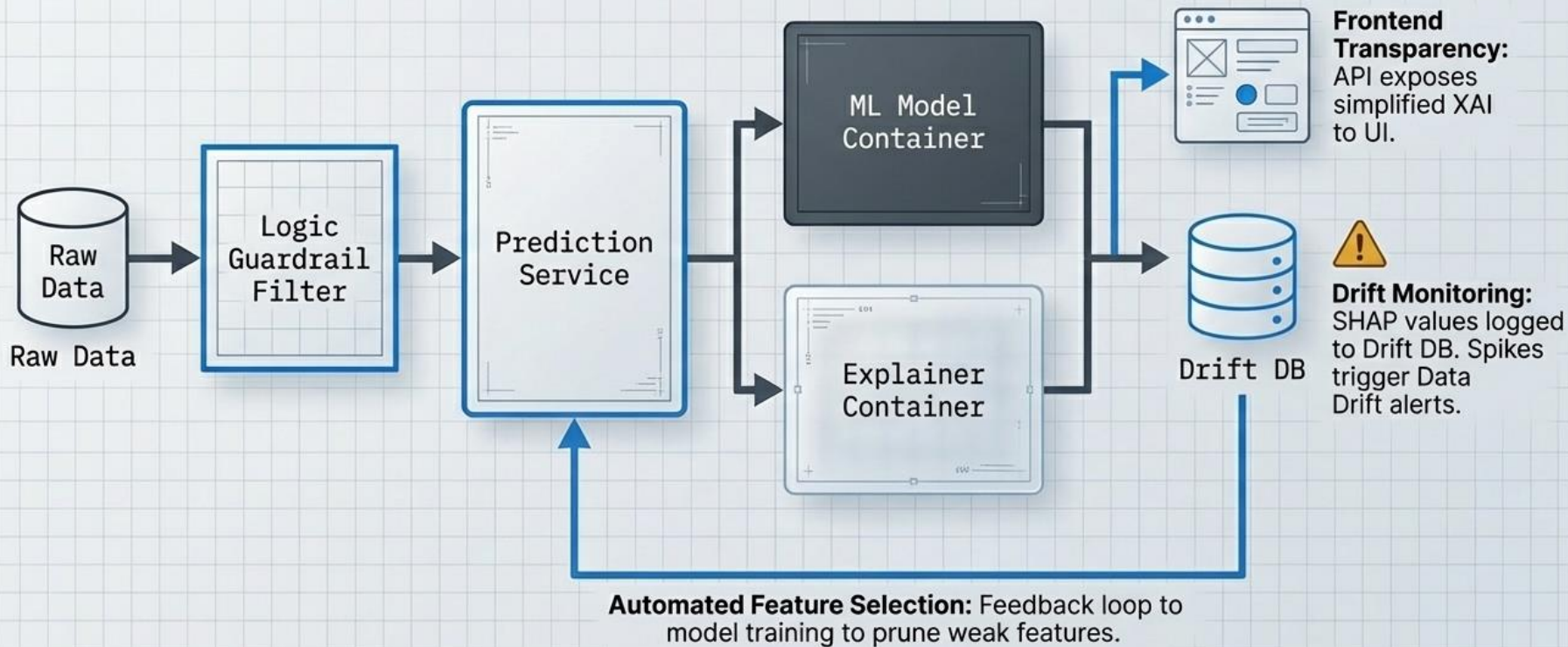


SHAP summary plot for feature importance at classification level 2

Architectural Decision Matrix: LIME vs. SHAP

	LIME	SHAP
Underlying Math	Local surrogate models (Linear)	Game theory (Shapley values)
Scope of Insight	 Local only	  Local and Global
Computational Cost	 Low / Fast	 Very expensive / Approximations needed
Consistency	 Can be unstable due to random perturbation	 Mathematically rigorous and consistent
Best System Use	Rapid local debugging	Pipeline auditing & feature selection

The Trust-Engineered Pipeline Blueprint



Summary & What's Next



Logic Rules

Use for absolute system guardrails and safety — deterministic, fast, always auditable.



LIME

Fast, local approximations. Perturb → observe → fit a local linear surrogate model.



SHAP

Unified, mathematically sound feature attribution via Shapley values. Local + Global.



Architecture

Engineer XAI into pipelines: feature selection, drift monitoring, UI transparency.